Software Requirements Specification

for

<MindScribe>

Version 1.0 approved

Prepared by <Muhammad Razi>

<FAST NUCES>

<March 06, 2026>

Table of Contents

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This document specifies the software requirements for Mindscribe, version 1.0. Mindscribe is an AI and RAG-powered application designed to make information storage and retrieval easier and more efficient. This document covers the major scope of the web-based system.

1.2 Document Conventions

This document follows standard IEEE SRS formatting. Priorities for higher-level system features are assumed to be inherited by their detailed requirements.

1.3 Intended Audience and Reading Suggestions

This document is intended for developers, project evaluators, and end-users testing the application. Readers should begin with the overall description in Section 2 before proceeding to the specific system features in Section 4.

1.4 Product Scope

Mindscribe solves the problem of ideas becoming fragmented and lost between various notes. It prevents the duplication of information and reduces the margin of error when listing details. By acting as a central, intelligent repository, the product has the goal of creating an efficient, automated note management system that formulates links and link strength between notes rather than relying on manual linking.

1.5 References

SE Project Proposal i.e Mindscribe, submitted to Ms. Mehroze Khan on 09/02/2026.

2. Overall Description

2.1 Product Perspective

Mindscribe is a web based application whose primary purpose is to handle context and relation between our information (notes) in a way, existing similar products like obsidian, notion will not.

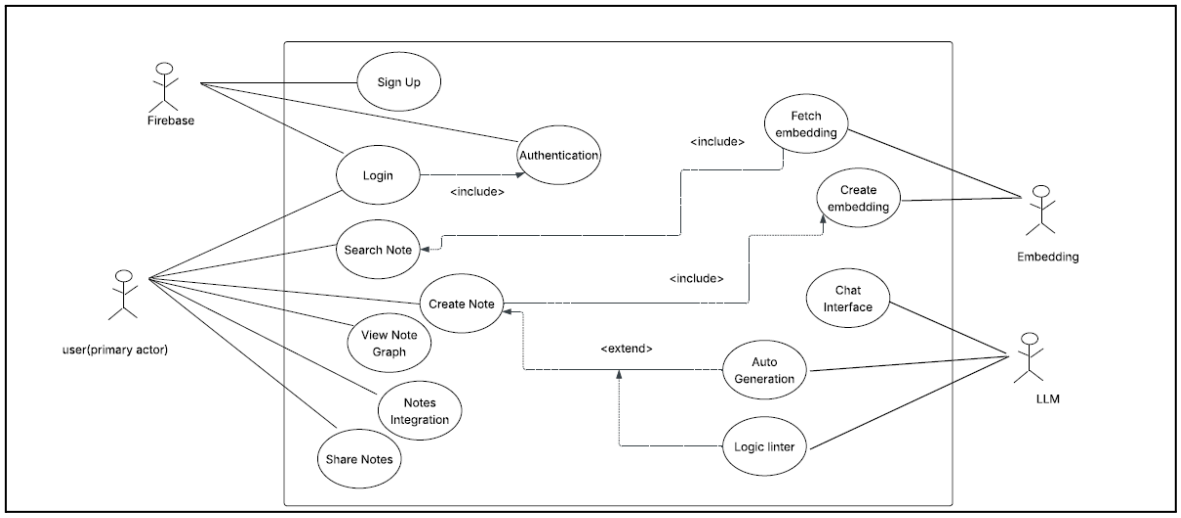
2.2 Product Functions

The major functions the product must perform include:

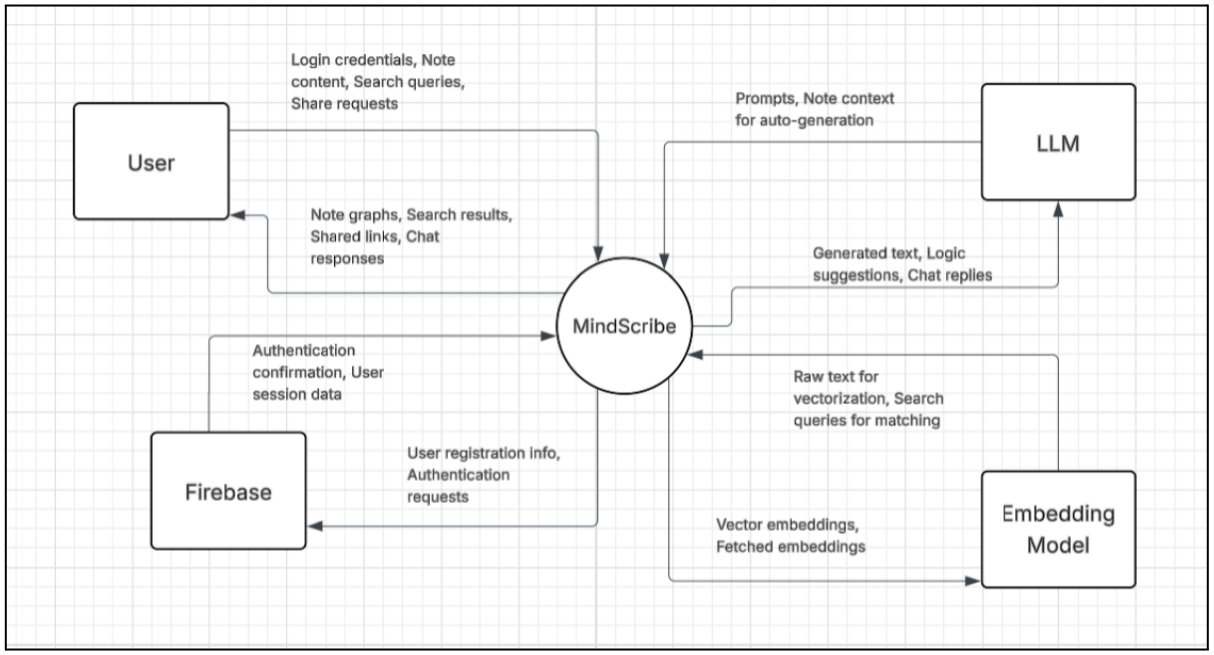
- ***Hybrid Searching:** Retrieval of notes via semantic processing or metadata.*

- ***Knowledge Graph:*** *Visual representation of notes based on similarity.*
- ***Critic & AutoGeneration:*** *Evaluation of written notes and auto-creation of notes based on user prompts.*
- ***Summarization:*** *Summarizing stored notes based on a user query topic.*
- ***Synthesizing:*** *Interacting with multiple note objects at a time for the purpose of synthesizing*
- ***Sharing:*** *Multiple users can share their note entries with other users*

Use Case Diagram:



DFD(Level 0):



2.3 User Classes and Characteristics

The user classes range from students, corporate workers and managers, and ordinary users. Majorly, it incorporates those users which requires efficient personal knowledge.The product is strictly designed for personal use and is not currently scaled or intended for enterprise-level deployment.

2.4 Operating Environment

The software will operate as a web-based application, requiring access via a modern web browser. It relies on a continuous and stable internet connection to communicate with the backend AI engines. The backend environment will be hosted on servers capable of running Python, processing Langchain/LangGraph workflows, and managing complex PostgreSQL databases.

2.5 Design and Implementation Constraints

The system architecture must strictly adhere to the following technological constraints:

Frontend: *Must be developed using React (JavaScript), as it is the standard for frontend projects.*

Backend: *Must be developed using FastAPI (Python) to support the heavy reliance on AI-based libraries.*

AI Engine: *Must utilize Langchain and LangGraph for processing logic.*

Database: *Must utilize PostgreSQL for storing metadata and a FAISS vector store for storing note embeddings. Furthermore, the pgvector PostgreSQL extension must be used for handling embeddings and metadata simultaneously.*

2.6 User Documentation

A video tutorial(screencast) demonstrating how to navigate this web app will be shared with users.

2.7 Assumptions and Dependencies

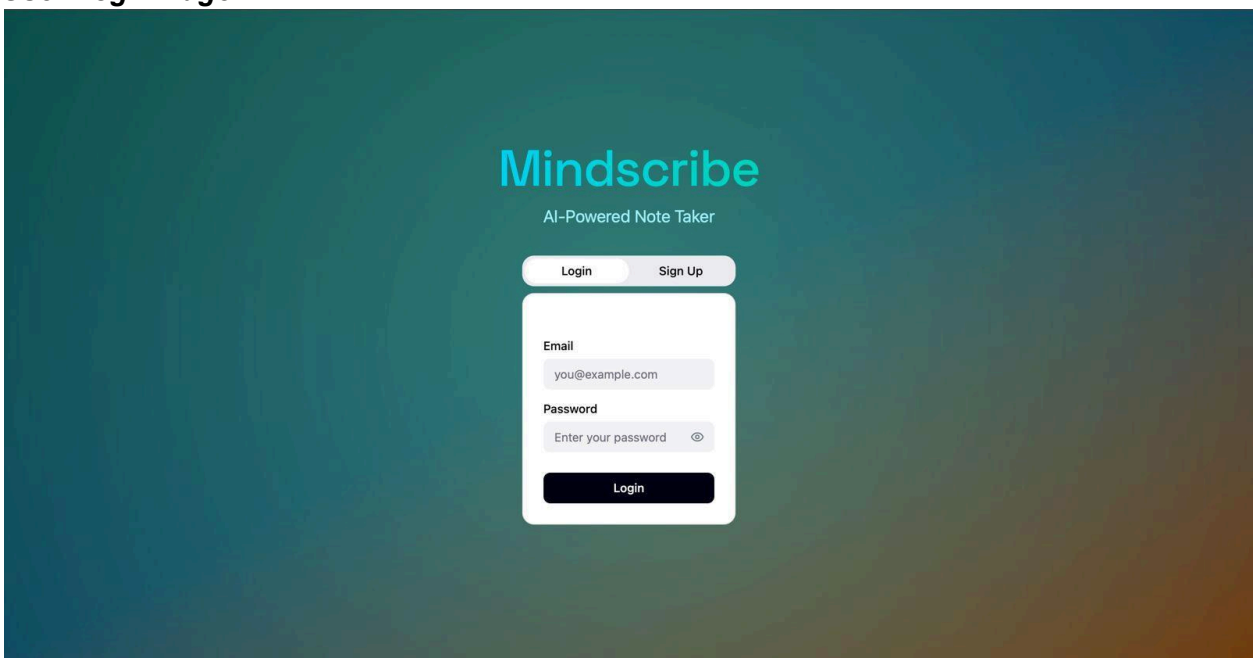
The primary assumption is that the user will maintain a stable internet connection, as local processing of LLMs and RAG workflows is not supported in this scope. The project is heavily dependent on the continued stability and availability of third-party libraries, specifically Langchain, LangGraph, FAISS, and the pgvector extension.

3. External Interface Requirements

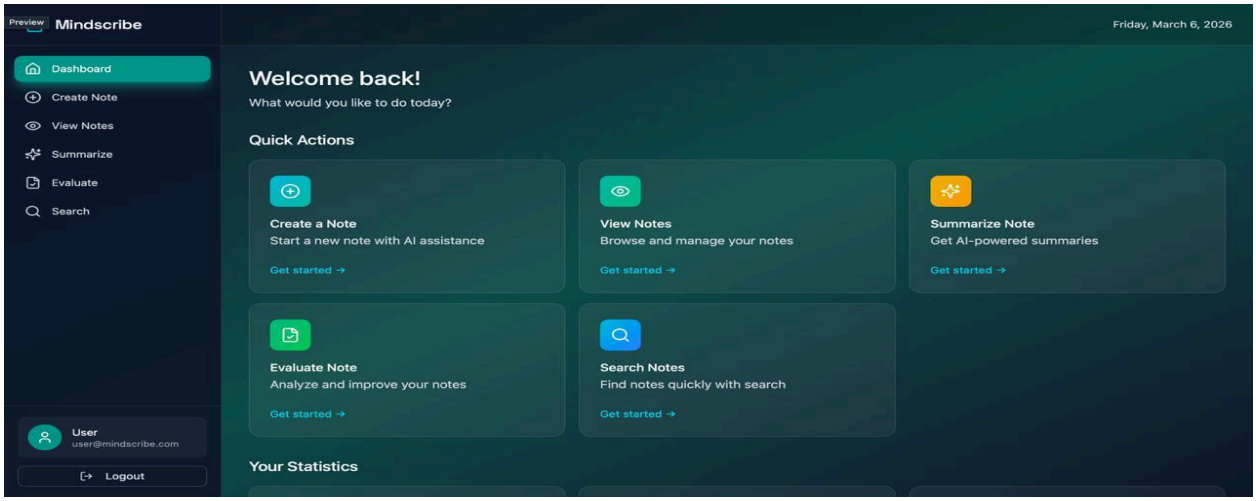
3.1 User Interfaces

The user interface, built with React, will provide a clean, web-based dashboard for text input and note management. The interface will consist of a primary text editor for note creation, a side-panel for the Context Radar to display related notes, and a dedicated canvas view for the interactive Knowledge Graph. Furthermore, it will provide a chat-like interface so the user can query chatbot interface for summarizing, synthesizing and for fetching general information about his own stored information.

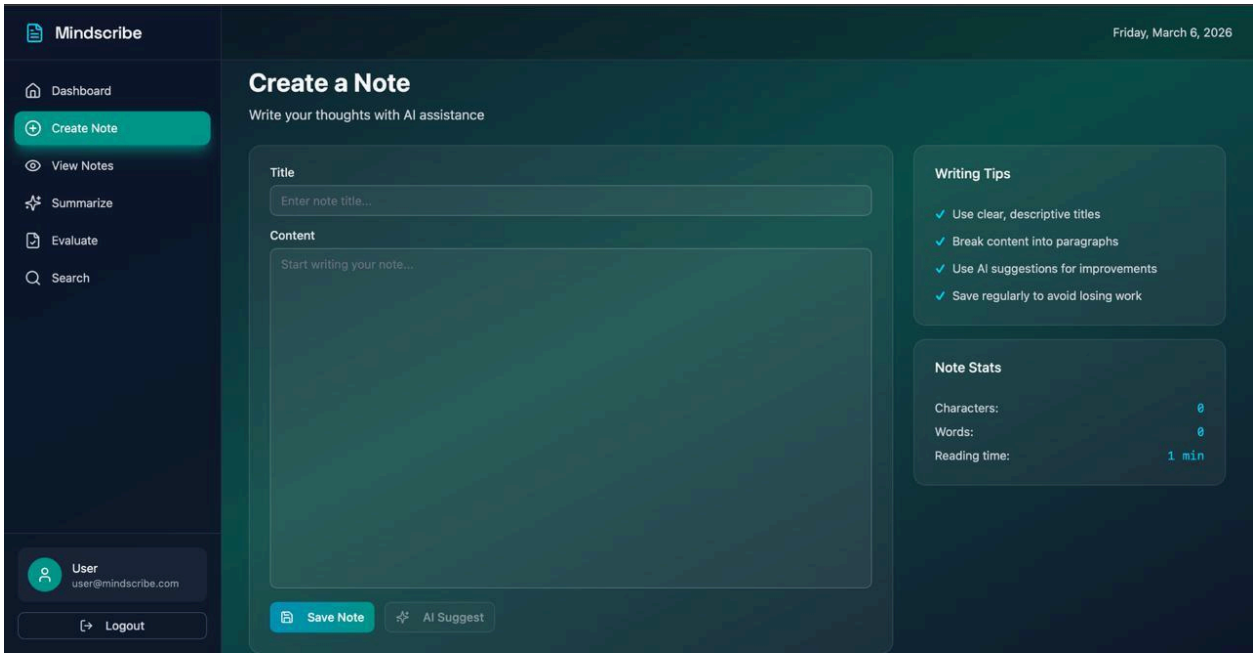
User Login Page:



User Dashboard:



Create Note:



3.2 Hardware Interfaces

As a web-based application utilizing standard text inputs, there are no specialized hardware interface requirements except standard peripherals (keyboard, mouse, monitor) used to navigate a web browser.

3.3 Software Interfaces

The React frontend will communicate exclusively with the FastAPI backend via standard RESTful API endpoints. The FastAPI backend will interface directly with the PostgreSQL database to read/write standard note metadata, and interface with the FAISS vector store to query high-dimensional embeddings. The backend will also interface with the Langchain/LangGraph frameworks to process user prompts and orchestrate the RAG pipelines.

3.4 Communications Interfaces

Communication between the user's web browser and the application servers will utilize standard HTTP/HTTPS protocols. Furthermore, Firebase services for authentication will be utilized

4. System Features

4.1 Hybrid Searching

4.1.1 Description and Priority

Allows for retrieval of notes using semantic processing (meaning-based) or metadata (attributes like date/title).

- **Priority**: High.

4.1.2 Stimulus/Response Sequences

- **Stimulus**: User enters a natural language query in the search bar.
- **Response**: System fetches embeddings and displays the most relevant notes.

4.1.3 Functional Requirements

- **REQ-1**: The system shall utilize a FAISS vector store to perform semantic similarity searches.
- **REQ-2**: The system shall allow users to filter results by note metadata via PostgreSQL.

4.2 Knowledge Graph

4.2.1 Description and Priority

A visual, interactive representation of notes where links are determined by content similarity.

- **Priority**: Medium.

4.2.2 Stimulus/Response Sequences

- **Stimulus**: User opens the Graph view.
- **Response**: System renders an interactive map of notes as nodes and relationships as edges.

4.2.3 Functional Requirements

- **REQ-3**: The system shall automatically calculate "link strength" between notes.
- **REQ-4**: The system shall provide a canvas-based UI for exploring the knowledge graph.

4.3 Summarization

4.3.1 Description and Priority

Generates a concise summary of multiple stored notes based on a specific user query.

- **Priority**: High.

4.3.2 Stimulus/Response Sequences

- **Stimulus**: User asks for a summary of a specific topic across all their notes.
- **Response**: System synthesizes relevant info and presents a single summary.

4.3.3 Functional Requirements

- **REQ-5**: The system shall allow users to summarize multiple similar notes at once.

4.4 AutoGeneration

4.4.1 Description and Priority

Automatically creates new note drafts based on specific user prompts.

- **Priority**: High.

4.4.2 Stimulus/Response Sequences

- **Stimulus**: User provides a topic or prompt to the AI engine.
- **Response**: System generates a full note and saves it to the database.

4.4.3 Functional Requirements

- **REQ-6**: The system shall use LangGraph/LangChain to orchestrate note generation.

4.5 Critic (Logic Linter)

4.5.1 Description and Priority

Evaluates written notes for clarity, identifies contradictions, and suggests structural improvements.

- **Priority**: Medium.

4.5.2 Stimulus/Response Sequences

- **Stimulus**: User triggers the "Critic" or "Linter" function on a specific note.
- **Response**: System identifies logical errors or missing details in the text.

4.5.3 Functional Requirements

- **REQ-7**: The system shall analyze note context to recommend a proper structure for ideas.
- **REQ-8**: The system shall flag contradictory points within a single note entry.

4.6 Memorization (User History)

4.6.1 Description and Priority

Maintains a history of user actions and created notes to improve the AI's contextual experience over time.

- **Priority**: Low.

4.6.2 Stimulus/Response Sequences

- **Stimulus**: User interacts with the system or creates new content.
- **Response**: System updates the user's profile/context to improve future responses.

4.6.3 Functional Requirements

- **REQ-9**: The system shall store interaction history to provide personalized retrieval.

4.7 Context Radar

4.7.1 Description and Priority

A dynamic list that displays existing notes related to the one currently being worked on.

- **Priority**: Medium.

4.7.2 Stimulus/Response Sequences

- **Stimulus**: User opens or starts editing a note.
- **Response**: System displays a sidebar with "related notes" based on real-time similarity.

4.7.3 Functional Requirements

- **REQ-10**: The side-panel shall update automatically as the content of the current note changes.

4.8 Sharing & Integration

4.8.1 Description and Priority

Allows users to share their note entries with other users and integrates with Firebase for secure access.

- **Priority**: Medium.

4.8.2 Stimulus/Response Sequences

- **Stimulus**: User clicks "Share" on a note entry.
- **Response**: System grants access to the specified user.

4.8.3 Functional Requirements

- **REQ-11**: The system shall use Firebase for user authentication and session management.
- **REQ-12**: The system shall support multi-user note sharing.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

1.System Availability:

The system shall aim for a 95% uptime during regular operating hours, acknowledging that it relies on a stable internet connection.

2.Search Response Time:

The system shall return hybrid search results (semantic and metadata-based) within 3 seconds under normal operating conditions

3.Concurrent Users:

The system shall support at least 50 concurrent users without any degradation in performance

4.Note rendering:

The system shall load and render an individual note, including its full text content, within 1 second of the user selecting it

5.AI Feature Response Time:

AI-driven features including AutoGeneration, Summarization, and Critic shall return results within 10 seconds. The Knowledge Graph shall render within 5 seconds for up to 50 linked notes.

5.2 Safety Requirements

1.Data Loss Prevention:

Because the system requires a stable internet connection, the frontend application shall automatically cache user input locally or trigger an auto-save every 30 seconds while writing a note to prevent data loss in the event of an unexpected network disconnection

2.AI Content Disclaimer:

The system shall clearly label all content produced via AutoGeneration, Summarization, or the Critic feature as AI-Generated to prevent the user from blindly relying on potentially hallucinated or inaccurate information

3.Account Deletion:

Upon user account deletion, the system shall permanently remove all associated notes, embeddings, and vector data from both PostgreSQL and the FAISS vector store within a defined timeframe.

4.Third-Party Data Handling:

The system shall only transmit the minimum necessary note content to third-party LLM and embedding APIs required to fulfill the user's request. Full note databases shall never be sent to external services in bulk.

5.Confirmation on Deletion:

The system shall prompt the user for explicit confirmation before permanently deleting a note.

5.3 Security Requirements

Authentication:

Access to any feature requires a verified session via Firebase.

• **Data Isolation:**

All notes, embeddings, and histories are strictly scoped to individual user accounts to prevent cross-account data influence.

• **Encryption**:

Communications between the React frontend, FastAPI backend, and AI engines must utilize standard HTTPS.

• **Privacy**:

Only the minimum necessary note content is sent to third-party APIs for processing; bulk database transfers are prohibited.

5.4 Software Quality Attributes

Availability:

The system targets 95% uptime during regular operating hours.

• **Reliability**:

To mitigate data loss from network issues, an auto-save or local cache must trigger every 30 seconds during note creation.

• **Transparency**:

AI-produced content (AutoGeneration, Summarization, Critic) must be clearly labeled as "AI-Generated" to address potential hallucinations.

• **Performance**:

Hybrid search results must return within 3 seconds, and individual notes must render within 1 second.

• **Usability**:

The interface must remain intuitive, featuring a real-time Context Radar side-panel and a canvas-based Knowledge Graph for visual exploration.

5.5 Business Rules

1.AI Feature Dependency:

AI-powered features (AutoGeneration, Summarization, Critic, Context Radar, Knowledge Graph) are only available when the user has an active internet connection. The system shall not attempt to run these features offline.

2.Authentication Requirement:

No system feature, including note viewing, shall be accessible without a verified and active user session. Unauthenticated users shall only be able to access the login and signup pages.

*3.**Account-Scoped Data:***

All notes, embeddings, and interaction history are strictly scoped to the user account under which they were created. Data from one account shall never influence the retrieval or AI outputs of another account.

*4.**Personal Use Only:***

The system is strictly intended for personal use. A single account may not be used as a shared workspace for teams or organizations. Enterprise-level deployment is explicitly out of scope for this version.

*5.**Note Ownership:***

Every note created within the system is owned by the user who created it. Only the owner has the authority to edit, delete, or share that note.

6. Other Requirements

Database Requirements:

The system must utilize PostgreSQL for metadata and a FAISS vector store for high-dimensional note embeddings.

• **Extension Requirements**:

The pgvector extension must be integrated within PostgreSQL to manage embeddings and metadata simultaneously.

• **Local Processing**:

There are no requirements for offline LLM or RAG workflows, as the system depends on third-party cloud libraries.

Appendix A: Glossary

• RAG (Retrieval-Augmented Generation): The AI architecture used to make information retrieval easier by referencing stored notes.

• FAISS: The vector store used to perform semantic similarity searches.

• LLM (Large Language Model): The external AI engine used for auto-generation, summarization, and logic linting.

• Semantic Search: Retrieval of notes based on meaning and context rather than just keyword matches.

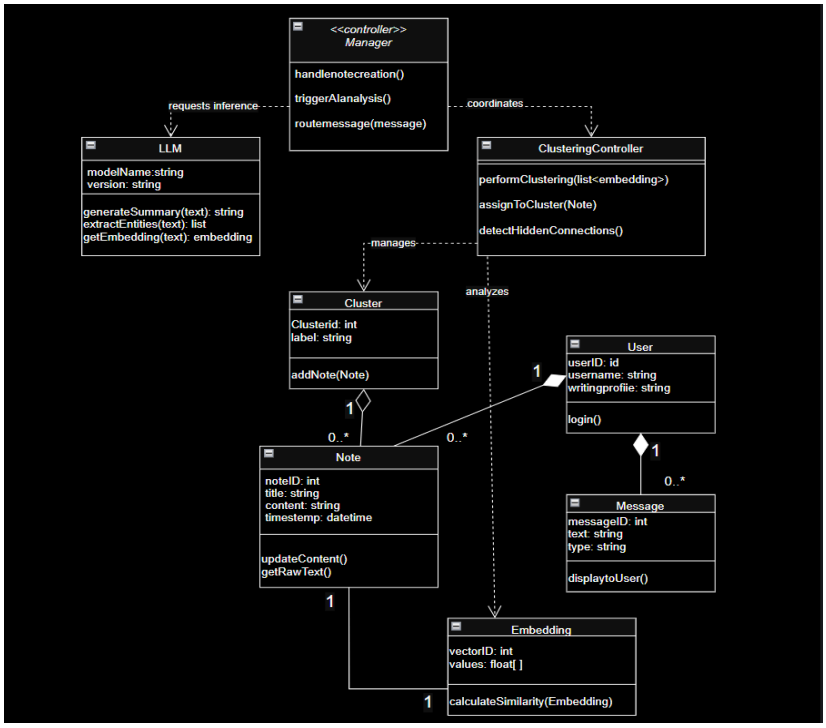
• Vector Embedding: High-dimensional numerical data used for matching search queries to relevant notes.

• Logic Linter: The "Critic" feature that evaluates written notes for clarity and identifies contradictions.

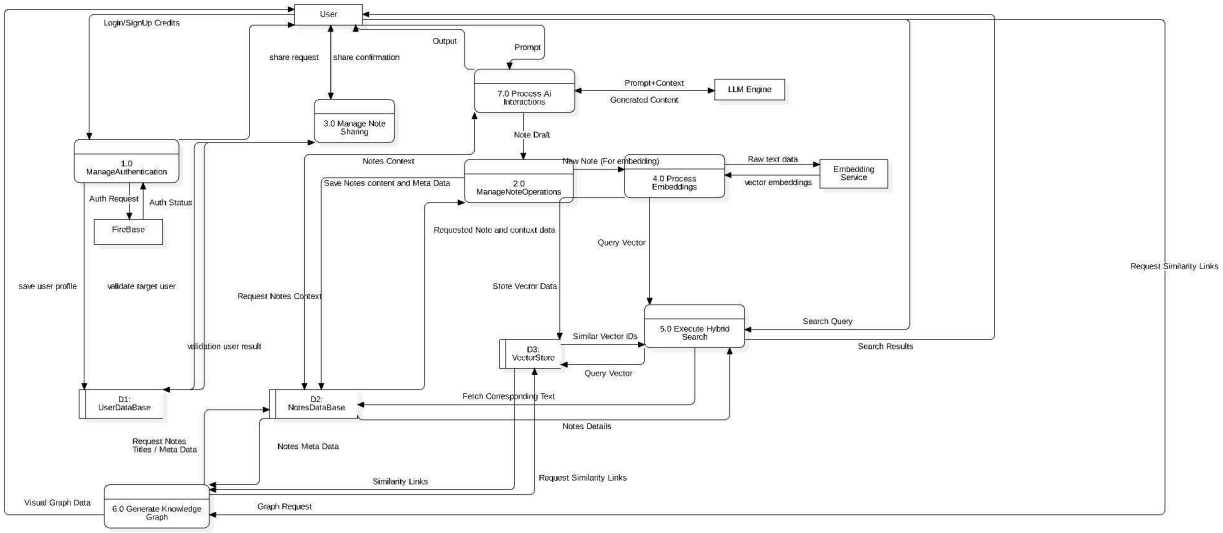
• Node/Edge: The visual components of the Knowledge Graph representing notes and their content similarity relationships.

Appendix B: Analysis Models

Class diagram:



DFD 1:



Appendix C: To Be Determined List

1. **Security Certifications**:

Specific privacy or security certifications (e.g., SOC2, GDPR compliance) that may be required for broader deployment.

2. **Account Deletion Timeframe**:

The exact "defined timeframe" in which data must be permanently purged from PostgreSQL and FAISS after a user deletes their account.

3. **Enterprise Scaling**:

Requirements and architecture changes needed if the product moves beyond the "Personal Use Only" restriction.

4. **Glossary Expansion:**

Additional organizational or project-specific acronyms to be added as development progresses.